

Departamento de Engenharia Informática
Licenciatura em Engenharia Informática e de Computadores
Licenciatura em Matemática Aplicada à Tecnologia e à Empresa
Licenciatura em Engenharia Informática, Redes e Telecomunicações

Segundo Trabalho Prático

Inteligência Artificial

Semestre de Verão 2025/2026

Version 1.00

Professor: Nuno Leite

Objetivos de aprendizagem

No final do **Segundo projeto prático**, os alunos deverão ser capazes de:

- ☐ Abordar a resolução de problemas como pesquisa, utilizando algoritmos básicos (não informados) e mais avançados (informados);
- ☐ Compreender o conceito de *espaço de estados*;
- ☐ Aplicar algoritmos de pesquisa não informada e informada para resolver o jogo *Sudoku* e sua variante (*Killer Sudoku*);
- ☐ Abordar a resolução de problemas como pesquisa num espaço de soluções;
- ☐ Compreender algoritmos de otimização que exploram um espaço de soluções, como o *simulated annealing* (SA) e *genetic algorithms* (GA);
- ☐ Aplicar *depth-first with iterative-deepening*, A*, SA e GA para resolver o jogo Sudoku.
- ☐ Aprofundar a prática de programação na linguagem Prolog.

Jogo Sudoku

Neste projeto, pretende-se desenvolver um *solucionador automático* para o famoso jogo Sudoku. A palavra Sudoku significa literalmente “dígito único”. Sudoku é um puzzle de lógica combinatória de posicionamento de números. O objetivo do jogo clássico é preencher uma grelha 9×9 com dígitos de tal forma que cada coluna, cada linha e cada uma das nove sub-grelhas 3×3 que compõem a grelha (também chamadas “quadrados”, “blocos” ou “regiões”) contenham todos os dígitos de 1 a 9. A soma de cada linha, coluna e sub-grelha deve ser igual a $45 = 1 + 2 + 3 + 4 + 5 + 6 + 7 + 8 + 9$. O criador do puzzle fornece uma grelha parcialmente preenchida com números fixos, que, para um puzzle bem formulado, tem uma única solução (baseado na Wikipedia¹).

A figura 1 ilustra um exemplo de um puzzle Sudoku (nível especialista) de <https://sudoku.com/pt/expert/>.

A sua tarefa é desenvolver programas (em Prolog e noutra linguagem à sua escolha) que demonstrem o comportamento de diferentes solucionadores, especificamente solucionadores que empregam os seguintes algoritmos:

- aprofundamento iterativo ou *depth-first with iterative-deepening* (não informado), programado

¹<https://en.wikipedia.org/wiki/Sudoku>

Dificuldade: Fácil Médio Difícil **Especialista** Diabólico

6			5					
		1			7			
					4	3		6
1	2	8						
	7					5	4	
	5					9		
			8	2				5
7				1			2	
							8	

Figure 1: Exemplo de um puzzle Sudoku (nível especialista).

em Prolog;

- pesquisa melhor primeiro (*best-first search*) ou A^* (informado), programado em Prolog. Deve elaborar uma heurística admissível adequada;
- *simulated annealing* – SA (algoritmo de otimização), programado numa linguagem à sua escolha;
- *algoritmo genético* – GA (algoritmo de optimização), programado numa linguagem à sua escolha.

Estes algoritmos encontram-se disponíveis no sítio GitHub da disciplina de Inteligência Artificial e estão escritos em Prolog e Matlab/Octave.

A entrada/saída é baseada em texto.

A representação do *estado/solução/cromossoma* (respetivamente, em busca no espaço de estados/SA/GA) deve ser cuidadosamente elaborada, resultando em operadores simples e eficientes.

Deve demonstrar o comportamento dos solucionadores automatizados para alguns níveis de exemplo, que vão desde puzzles simples a puzzles muito difíceis (todos os puzzles devem ser bem formulados). Não precisa de implementar carregadores de níveis (os níveis podem ser codificados directamente para simplificar), mas se desejar, pode utilizar o formato de níveis publicado em: <https://github.com/t-dillon/tdoku/blob/master/data.zip>. Neste URL, também poderá obter uma base de dados de problemas Sudoku. Deverá testar com problemas de diferentes complexidades.

Variante Killer Sudoku

Depois de implementar o Sudoku original, deverá adaptar o seu código para suportar problemas Killer Sudoku.

O Killer Sudoku é uma variante do Sudoku. No Killer Sudoku, existe um novo conceito de “gaiola” ou *cage* – um grupo de células dentro de uma linha pontilhada que, em conjunto, formam uma soma representada por um pequeno número acima. Tal como no Sudoku, o seu objetivo final é preencher a grelha com números, tendo em atenção a “regra da soma”, ou seja, é necessário garantir que a soma dos números em cada casa é igual ao número no canto superior esquerdo da casa.

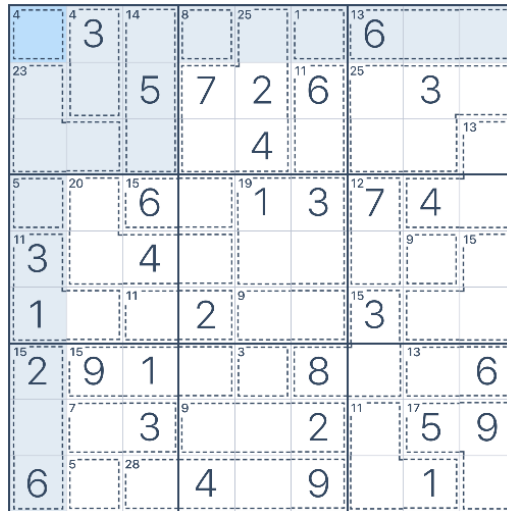


Figure 2: Um exemplo de um nível de jogo *Killer Sudoku*.

Opcional

Poderá implementar técnicas não lecionadas nas aulas mas que possam melhorar o desempenho dos algoritmos. São exemplos as técnicas de *reheating* e aceleração no SA.

Data de entrega: 19 de maio de 2026 até às 23:59.

A entrega do trabalho deve incluir o relatório, o código Prolog (solucionador utilizando algoritmos de pesquisa) e outros códigos desenvolvidos (solucionador utilizando algoritmos de otimização, *simulated annealing* e *algoritmos genéticos*), disponibilizados no sistema Moodle. O relatório deve ser conciso, explicar os principais predicados/estrutura e justificar todas as decisões tomadas. Deve indicar a composição do grupo de alunos e a informação da unidade curricular.

O relatório deve apresentar resultados experimentais em diversos tipos de puzzles e uma comparação do tempo de execução dos solucionadores.